

Bài 08:

Lập trình đa tuyến & ứng dụng trong mô hình client - server

Mô hình tuần tự

```
SocketServer ss = new SocketServer(2000);
while(true)
{
    Socket s = ss.accept();
    Scanner sc = new Scanner(s.getInputStream());
    PrintWriter p = new PrintWriter(s.getOutputStream());
    while (s != null) {
        int ev = sc.nextInt();
        switch (ev) {
            case 0:
                s.close();
                s = null;
                break;
            case 1: //xử lý các giao thức giao tiếp khác
        }
    }
    //tiếp tục lập giao tiếp với client khác
}
```

Mô hình đa tiến trình

```
SocketServer ss = new SocketServer(2000);  
while (true) {  
    Socket s = ss.accept();  
    //tạo ra một tiến trình riêng xử lý cho client mới kết nối tới  
    SocketHandler sh = new SocketHandler(s);  
    //tiếp tục giao tiếp với client khác  
}
```

MultiThread Programming

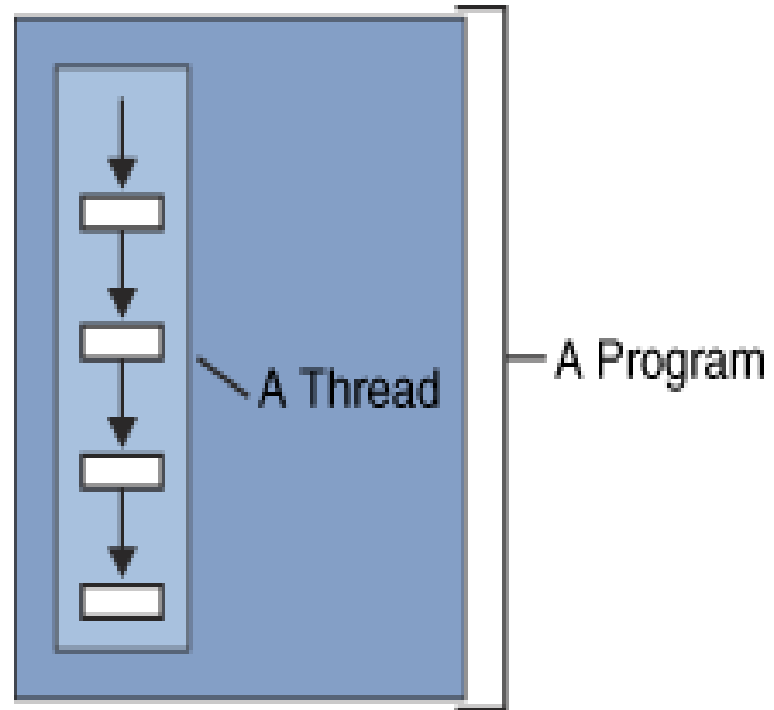
- ❑ Khái niệm về thread
- ❑ Tạo ra các thread
 - Lớp Thread
 - Cài đặt lớp Runnable
- ❑ Vòng đời của một Thread
- ❑ Lập lịch cho thread
- ❑ Đồng bộ hóa các thread

Khái niệm về Thread

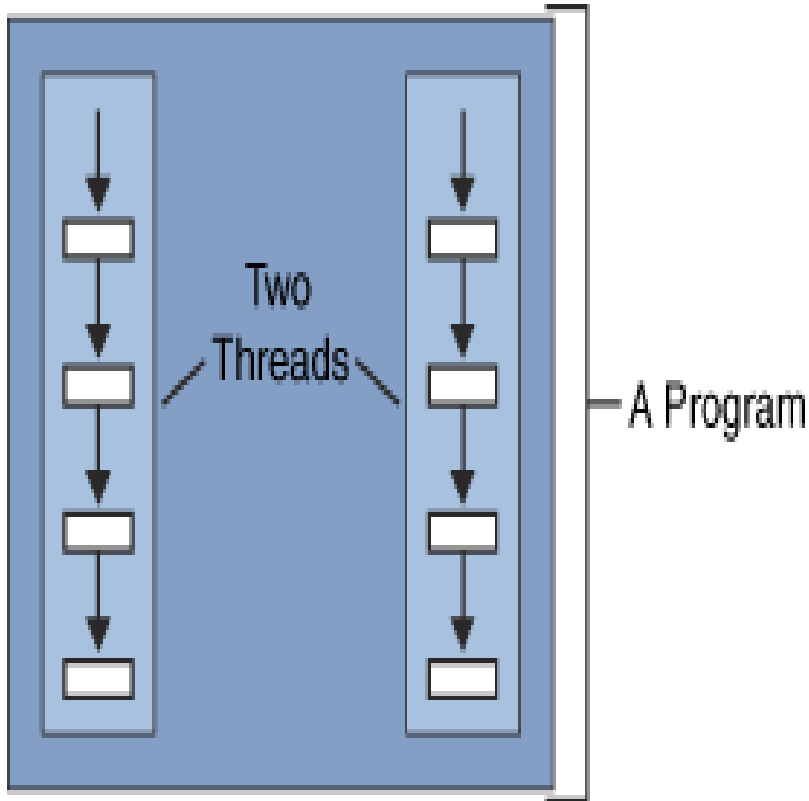
- ❑ Chương trình tuyến tính: Có một khởi đầu
 - Chuỗi các hành động với thứ tự xác định
 - Một điểm kết thúc
 - Chỉ có duy nhất một lệnh thực thi tại một thời điểm

Khái niệm về thread

- ❑ Một thread cũng bao gồm các đặc điểm của các chương trình tuyến tính.
- ❑ Tuy vậy, bản thân thread **không phải là một chương trình**: Nó không thể tự chạy, mà phải thuộc vào một chương trình



Thread: Định nghĩa



- ❑ Một *thread* là một dòng điều khiển tuần tự bên trong một chương trình.
- ❑ Lý do sử dụng thread:
 - Trong một chương trình có thể có nhiều *thread* chạy đồng thời, thực thi các tác vụ khác nhau.

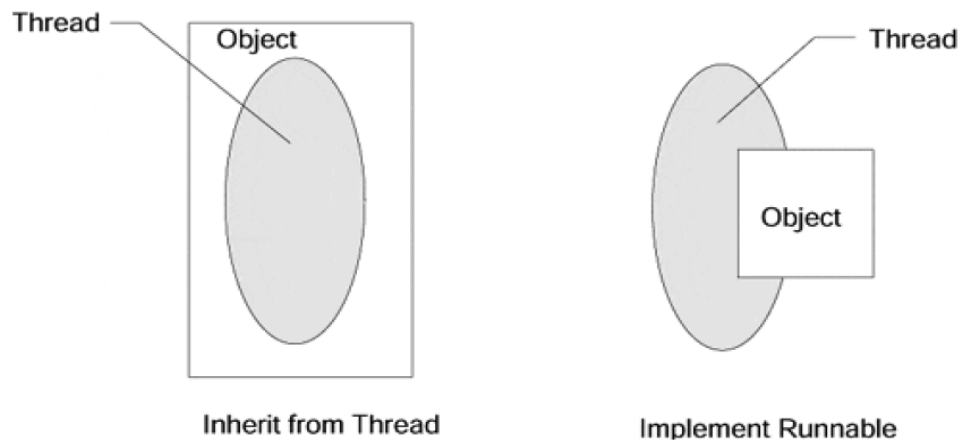
Đặc điểm của thread

- ❑ Sử dụng nguồn tài nguyên được phân phát cho từng tiến trình và nguồn tài nguyên chung trong chương trình(đồng bộ hóa tiến trình).

Tạo Thread trong Java

- ❑ Java cung cấp một giao diện nhất quán cho các thread. Để cài đặt thread:
 - Cài đặt giao diện Runnable
 - Dẫn xuất từ lớp Thread,

Figure 1-5. Thread controls are accessed from different locations depending on the method chosen.



Dẫn xuất từ lớp Thread

- ❑ Cách đơn giản nhất để tạo ra các Thread là tạo lớp con từ lớp Thread
 - Lớp mới tạo có đầy đủ cơ chế của một thread
 - Ta phải nạp chồng phương thức Run của lớp Thread.
 - Phương thức `run()` chứa đoạn mã sẽ được thực hiện tuần tự, nhưng “đồng thời” với các thread khác trong một chương trình.

Phương thức khởi tạo lớp Thread

Constructor	Ý nghĩa
Thread()	Tạo Thread với giá trị mặc định
Thread(String name)	Tạo Thread với tên cho trước
Thread(Runnable target)	Tạo thread liên kết với đối tượng Runnable
Thread(Runnable tar, String name)	Tạo Thread liên kết với đối tượng Runnable với tên cho trước

Thực thi Thread

- ❑ Để thực thi thread, ta gọi phương thức `void start()`. Khi được thực thi, thread sẽ chạy các lệnh trong phương thức `run()`.
- ❑ Lưu ý: Nếu thread đã `start()` rồi, việc gọi `start()` lần nữa sẽ bị phát sinh exception *`IllegalThreadStateException`*. Để `start()` lại thread, ta cần tạo mới thread.

Ví dụ

```
public class SimpleThread extends Thread
{
    public SimpleThread(String str)
    {super(str);}

    public void run()
    {
        for (int i = 0; i < 10; i++){
            System.out.println(i + " " + this.getName());
            try {
                sleep((long) (Math.random() * 1000));
            } catch (InterruptedException e){}
        }
        System.out.println("DONE! " + getName());
    }
}
```

- ❑ Chương trình sau đây chỉ nhằm mục đích minh họa hoạt động của hai Thread thay phiên nhau.
- ❑ Mặc dù, như chúng ta thấy, các lệnh khởi tạo chúng được viết tuần tự. Ta có thể thêm vào các thread khác nếu muốn.

```
public class TwoThreadsDemo
{
    public static void main(String[] args) {
        new SimpleThread("Dalat").start();
        new SimpleThread("Sapa").start();
    }
}
```

Phương thức *run* của một Thread

- ❑ Là phương thức ảo (virtual method)
- ❑ Hoạt động của Thread sẽ kết thúc ngay khi thoát khỏi phương thức *run*
 - Cần thiết lập các điều kiện dừng cho thread trong phương thức *run*
 - Nếu không, để dừng thread, chúng ta cần phải cầu viện đến các phương thức khác như *stop* hoặc *destroy*

Giao diện Runnable

- ❑ Nếu không thể tạo class kế thừa từ Thread, ta có thể cho nó implement interface Runnable.
- ❑ Khi đó, class phải hiện thực phương thức void run() của interface này. Sau đó, ta tạo thread liên kết với Runnable thông qua constructor tương ứng của lớp Thread.
- ❑ Khi thread được start, nó sẽ gọi thực thi lệnh trong phương thức run() của Runnable liên kết với nó.

Ví dụ 1

```
14 public class MyRunnableEx1 {
15
16     public static void main(String[] args) {
17         Runnable r1 = new MyRunnable();
18         Runnable r2 = new MyRunnable();
19         Thread t1 = new Thread(r1);
20         Thread t2 = new Thread(r2);
21         t1.start();
22         t2.start();
23     }
24 }
25
26 class MyRunnable implements Runnable {
27
28     @Override
29     public void run() {
30         int i = 0;
31         while (i < 10) {
32             System.out.println(i);
33             i++;
34             try {
35                 sleep(1000);
36             } catch (InterruptedException ie) {
37             }
38         }
39     }
40 }
```

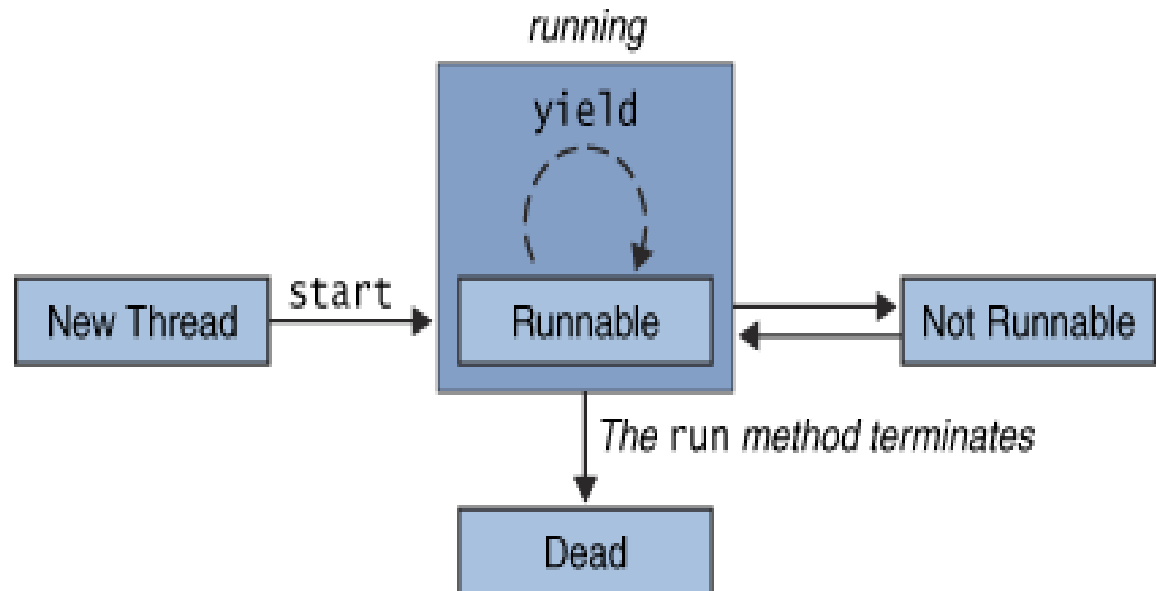
Ví dụ 2: Cài đặt lớp ClockThreadApplication

```
16 public class ClockThreadApplication implements Runnable {
17     private Thread clockThread = null;
18
19     public ClockThreadApplication() {
20         clockThread = new Thread(this, "Clock");
21         clockThread.start();
22     }
23
24     private String getTime() {
25         Calendar cl = Calendar.getInstance();
26         Date date = cl.getTime();
27         DateFormat dateFormatter = DateFormat.getTimeInstance();
28         return dateFormatter.format(date);
29     }
30
31     @Override
32     public void run() {
33         Thread myThread = Thread.currentThread();
34         while (clockThread == myThread) {
35             System.out.println(getTime());
36             try {
37                 Thread.sleep(1000);
38             } catch (InterruptedException ie) {
39                 ie.printStackTrace();
40             }
41         }
42     }
43
44     public static void main(String[] args) {
45         ClockThreadApplication clock = new ClockThreadApplication();
46     }
47 }
```

Quản lý Thread (Thread Control)

❑ Lớp Thread cung cấp cho người lập trình các công cụ để quản lý:

- 1. start
- 2. stop
- 5. sleep
- 6. destroy
- 7. yield
- 8. run
- 9. isAlive



Cẩn thận!

- ❑ *Phải chắc chắn rằng mọi thread đã tạo phải được dừng lại (hoặc chính nó dừng lại hoặc đã bị dừng lại bởi tiến trình cha). Ngược lại, bạn rất có thể phải có vài thread đang chạy và đang tiêu tốn tài nguyên sau khi mà ứng dụng chứa nó đã dừng.*

Dừng một Thread

- ❑ Phương thức `stop` được sử dụng để dừng một Thread vào bất cứ lúc nào. Tuy vậy, nó không an toàn.
- ❑ Cách làm được khuyến khích là để một Thread “chết tự nhiên”, như phương thức `run` sau:

```
public void run() {  
    int i = 0;  
    while (i < 100) {  
        i++;  
        System.out.println("i = " + i);  
    }  
}
```

Thoát an toàn khỏi Thread (2)

- Xét lại phương thức `run` của lớp `ClockThreadApplication`, để ý hoạt động của nó và điều kiện dừng của vòng lặp

```
//the run method of the clockThread
public void run() {
    Thread myThread = Thread.currentThread();

    while (clockThread == myThread) {
        System.out.println(getTime());
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {}
    }
}
```

Thoát an toàn khỏi Thread (4)

- Đây là một cách sửa đổi đơn giản với phương thức run của lớp ClockThreadApplication.

```
//the run method of the clockThread
public void run() {
    Thread myThread = Thread.currentThread();

    while (clockThread == myThread) {
        System.out.println('\n' + getTime());
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {}

        if (countTimer++ == 10)
            clockThread = null;
    }
}
```

Thoát an toàn khỏi Thread (5)

- ❑ Đối với phiên bản đồ họa, điều này đơn giản hơn với việc xử lý sự kiện đóng ứng dụng `SwingApplication`.

```
public void windowClosing(WindowEvent e) {  
    clockThread = null;  
    System.exit(0);  
}
```

Trạng thái Not Runnable (1)

- ❑ Một thread trở thành Not Runnable khi một trong các điều kiện sau xuất hiện:
 - Phương thức *sleep* của nó được gọi.
 - Thread gọi phương thức *wait* để chờ thỏa một điều kiện cụ thể.
 - Thread bị chặn trên các thiết bị nhập/xuất.

Trạng thái Not Runnable (2)

- ❑ `clockThread` trong ứng dụng `ClockThreadApplication` trở nên Not Runnable khi phương thức run gọi `sleep` trên thread

```
//the run method of the clockThread
public void run() {
    Thread myThread = Thread.currentThread();

    while (clockThread == myThread) {
        System.out.println(getTime());
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {}
    }
}
```

Trạng thái Not Runnable (3)

- ❑ Một Thread trong trạng thái Not Runnable sẽ không tiêu tốn bất kỳ nguồn tài nguyên hệ thống nào, ngay cả CPU lúc đó đang rỗi.
- ❑ Vào và thoát khỏi trạng thái Not Runnable
 - Nếu một thread đã được đặt vào trạng thái ngủ, nó phải đợi trong thời gian bằng số milli giây đã chỉ định.
 - Nếu một thread đang đợi một điều kiện, thì thread khác phải thông báo với thread đang chờ về sự thay đổi với phương thức `notify` hoặc `notifyAll`.
 - Nếu một thread bị khóa trên I/O, thao tác I/O phải hoàn tất.

MultiThread Programming

- ❑ Khái niệm về thread
- ❑ Tạo ra các thread
- ❑ Vòng đời của một Thread
- ❑ Lập lịch cho thread
- ❑ Đồng bộ hóa các thread

Xử lý đồng thời Thread ?

- ❑ Có phải tất cả các Thread đều đang chạy?
- ❑ Khi một thread đang chạy, nó sử dụng nguồn tài nguyên cho riêng nó.
- ❑ Nếu có nhiều Thread "đang cùng chạy" trên một máy chỉ có một CPU, JVM phải cài đặt một biểu đồ lập lịch để chia sẻ CPU.
- ❑ Trên thực tế, **không thể tồn tại** tình huống mà có nhiều hơn một thread đang cùng chạy. Đó chỉ đơn giản là sự giả lập. Một thread "đang chạy" phải dừng các hoạt động để chờ đến lượt nó sử dụng CPU.

Lập lịch cho Thread

- ❑ Sự thực thi của nhiều Thread trên một CPU duy nhất theo một thứ tự nào đó gọi là *lập lịch* (schedule)
- ❑ Java hỗ trợ thuật toán lập lịch đơn giản : "*Theo độ ưu tiên cố định*" (*fixed-priority scheduling*).
- ❑ JVM lên lịch để chạy các Thread Runnable chủ yếu dựa trên độ ưu tiên của nó.
- ❑ *Độ ưu tiên của Thread* là một số nào đó nằm giữa MIN_PRIORITY và MAX_PRIORITY (các hằng số định nghĩa trong lớp Thread, gồm
 - MAX_PRIORITY: 10
 - MIN_PRIORITY: 0
 - NORM_PRIORITY: 5, default value).

Lập lịch cho Thread

- ❑ Khi một Thread mới được tạo ra, nó kế thừa độ ưu tiên từ Thread đã tạo ra nó.
- ❑ Để sửa đổi độ ưu tiên của Thread, dùng phương thức `setPriority`.
- ❑ Ví dụ:

```
t.setPriority(8);
```

Lập lịch cho Thread

- ❑ Thread được chọn để chạy, sẽ dừng lại khi có một trong các điều kiện sau:
 - Một thread có độ ưu tiên cao hơn trở thành runnable.
 - Nó nhường quyền sử dụng CPU (gọi phương thức `yields`), hoặc phương thức `run` của nó thoát.
 - Trên các hệ thống có hỗ trợ phân chia thời gian (time-slicing), và thời gian được phân cho nó đã hết.
- ❑ Sau đó thread thứ hai được trao quyền để chạy, và cứ thế, cho đến khi bộ thông dịch kết thúc.

Sự từ bỏ CPU

- ❑ Nói chung nên cố gắng viết các thread biết-cư-xử; biết nhường CPU một cách định kỳ và để các thread khác có cơ hội để chạy.
- ❑ Thread nhường CPU một cách tự nguyện bằng cách gọi phương thức yield.
- ❑ Phương thức yield trao cho các thread khác có cùng độ ưu tiên một cơ hội để chạy. Nếu không có thread nào có cùng độ ưu tiên đang trong trạng thái Runnable, yield được bỏ qua.

MultiThread Programming

- ❑ Khái niệm về thread
- ❑ Tạo ra các thread
- ❑ Vòng đời của một Thread
- ❑ Lập lịch cho thread
- ❑ Đồng bộ hóa các thread

Đồng bộ hóa cho phương thức

- ❑ Nếu nhiều thread cùng truy cập vào một vùng nhớ, lỗi có thể xảy ra. Để chỉ cho phép trong một lúc chỉ có tối đa một thread truy cập vào vùng nhớ, ta cần đồng bộ hóa bằng cách dùng từ khóa synchronized.

Ví dụ 1 (Trang 33)

```
13 public class MyThreadEx2 {
14     public static void main(String[] args) {
15         MyThread2 T[] = new MyThread2[3];
16         for (int i = 0; i < T.length; i++) {
17             T[i] = new MyThread2();
18             T[i].start();
19         }
20     }
21 }
22
23 class MyThread2 extends Thread {
24
25     public static int x = 5;
26
27     public static void Trans() {
28         if (x > 0) {
29             try {
30                 sleep(100);
31             } catch (InterruptedException ie) {
32             }
33             x--;
34         }
35     }
36
37     @Override
38     public void run() {
39         do {
40             Trans();
41             System.out.println(getName() + ": x=" + x);
42         } while (x > 0);
43     }
44 }
```

Ví dụ 2

```
13 public class MyThreadEx3 {
14     public static void main(String[] args) {
15         MyThread3 T[] = new MyThread3[3];
16         for (int i = 0; i < T.length; i++) {
17             T[i] = new MyThread3();
18             T[i].start();
19         }
20     }
21 }
22
23 class MyThread3 extends Thread {
24
25     public static int x = 5;
26
27     public static synchronized void Trans() {
28         if (x > 0) {
29             try {
30                 sleep(100);
31             } catch (InterruptedException ie) {
32             }
33             x--;
34         }
35     }
36
37     @Override
38     public void run() {
39         do {
40             Trans();
41             System.out.println(getName() + ": x=" + x);
42         } while (x > 0);
43     }
44 }
```

Lệnh synchronized

- Lệnh synchronized cho phép đồng bộ hóa một đối tượng mà không cần yêu cầu bạn tác động một phương thức synchronized trên đối tượng đó. Cú pháp:

```
synchronized (expr)
{statement}
```

- Khi có được khóa, statement được xử lý giống như nó là một phương thức synchronized trên đối tượng đó. Ví dụ: Chuyển các phần tử trong mảng thành các số không âm:

```
public static void abs(int[] v){
    synchronized(v){
        for(int i=0;i<v.length;i++){
            if(v[i]<0) v[i]=-v[i];
        }
    }
}
```

Ứng dụng: BAI08-BT01

- ❑ Phần này chúng ta xem xét một ứng dụng Producer-Consumer đơn giản
- ❑ Tham gia vào ứng dụng của chúng ta gồm có:
 - Một nhà sản xuất, đối tượng: **Producer**
 - Một khách hàng tiêu thụ, đối tượng: **Consumer**
 - Một đối tượng trung gian đóng vai trò nhà phân phối, đối tượng: **CubbyHole**
- ❑ Hoạt động: nhà sản xuất sẽ sinh một số, giao cho nhà phân phối lưu giữ trong kho của anh ta (phương thức **put**). Bên tiêu thụ sẽ nhận số có trong kho này (phương thức **get**). Yêu cầu bên tiêu thụ mỗi số đúng một lần, và không bỏ sót.

Lớp CubbyHole

```
13 public class CubbyHole {
14     private int contents;
15
16     public int get(int who) {
17         System.out.println("Customer #" + who + " got: " + contents);
18         return contents;
19     }
20
21     public void put(int who, int value) {
22         contents = value;
23         System.out.println("Producer #" + who + " put: " + contents);
24     }
25 }
```

Lớp Producer

```
13 public class Producer extends Thread {  
14     private CubbyHole cub;  
15     private int num;  
16  
17     public Producer(CubbyHole cub, int num) {  
18         this.cub = cub;  
19         this.num = num;  
20     }  
21  
22     @Override  
23     public void run() {  
24         for (int i = 0; i < 10; i++) {  
25             cub.put(num, i);  
26             try {  
27                 sleep((int) (Math.random() * 1000));  
28             } catch (InterruptedException ie) {  
29             }  
30         }  
31     }  
32 }
```

Lớp Consumer

```
13 public class Consumer extends Thread {  
14     private CubbyHole cub;  
15     private int num;  
16  
17     public Consumer(CubbyHole cub, int num) {  
18         this.cub = cub;  
19         this.num = num;  
20     }  
21  
22     @Override  
23     public void run() {  
24         int value = 0;  
25         for (int i = 0; i < 10; i++) {  
26             value = cub.get(num);  
27         }  
28     }  
29 }
```

Ứng dụng demo

```
public class ThreadSynchronization {  
13     public static void main(String[] args) {  
14         CubbyHole cub = new CubbyHole();  
15         new Consumer(cub, 1).start();  
16         new Producer(cub, 1).start();  
17     }  
18 }
```

Và cuộc chơi bắt đầu ...

```
Customer #1 got: 0
Producer #1 put: 0
Customer #1 got: 0
Customer #1 got: 0
Customer #1 got: 0
Customer #1 got: 0
Customer #1 got: 0
Customer #1 got: 0
Customer #1 got: 0
Customer #1 got: 0
Customer #1 got: 0
Producer #1 put: 1
Producer #1 put: 2
Producer #1 put: 3
Producer #1 put: 4
Producer #1 put: 5
Producer #1 put: 6
Producer #1 put: 7
Producer #1 put: 8
Producer #1 put: 9
```

Kết quả có đáp ứng yêu cầu đề bài?
Tại sao?

Điều kiện tranh đua

- ❑ Vấn đề chúng ta vừa gặp thường được gọi là vấn đề tranh đua (race condition).
- ❑ Một *race condition* là một tình huống mà trong đó hai hoặc nhiều hơn các thread hoặc tiến trình đọc hoặc ghi các dữ liệu chia sẻ, và kết quả cuối cùng tùy thuộc vào thời gian biểu mà theo đó các thread được lập lịch. Các điều kiện tranh đua có thể dẫn đến các kết quả không đoán trước và các lỗi rất khó phát hiện.

Giải pháp - sự đồng bộ hóa

- ❑ Các giải pháp đồng bộ hóa có thể:
 1. Cả producer và consumer phải thống nhất với nhau về cách đặt số. Một số sẽ chỉ được lấy ra/đặt vào trong một số điều kiện nào đó (nghe phức tạp quá!).
 2. Tổ chức lại lớp CubbyHole như một hàng đợi (không tồi chút nào, nhưng nếu tôi muốn tiêu thụ một số ngay khi nó được sinh ra kia)
 3. Sử dụng cơ chế đồng bộ hóa của Java với **synchronized**

Đồng bộ hóa trong Java

- ❑ Cơ chế đồng bộ hóa trong Java cung cấp một tập các công cụ để điều khiển các threads đồng thời chia sẻ một nguồn tài nguyên chung (file hoặc hàng đợi sự kiện chẳng hạn) theo cách tốt đẹp.
 - `synchronized` Không cho phép cả hai Thread đồng thời truy cập vào một đối tượng (đối tượng bị khóa bởi Thread “nhanchân”)
 - Các Thread đang đợi một điều kiện (`wait`) phải được thông báo rằng nó xong việc (qua `notify` và `notifyAll`)

Khóa một đối tượng

- ❑ Từ khóa `synchronized` đặt trong phần khai báo một block (có thể là toàn bộ phương thức) sẽ làm cho đoạn này trở thành *đoạn tới hạn* (critical section).
 - Khi có một đối tượng truy cập vào một đoạn tới hạn trong một đối tượng, nó sẽ khóa tất cả các đoạn tới hạn khác
 - vì vậy, ta sẽ khai báo hai phương thức `put` & `get` là các đoạn tới hạn trong lớp `CubbyHole`.

Khóa một đối tượng

```
1 public class CubbyHole {
2     private int contents;
3
4     public synchronized int get(int who) {
5         System.out.println("Consumer #" +
6                             who + " got: " + contents);
7         return contents;
8     }
9
10    public synchronized void put(int who, int value) {
11        contents = value;
12        System.out.println("Producer #" +
13                            who + " put: " + contents);
14    }
15 }
```

Với khai báo trên, khi Producer gọi phương thức put của CubbyHole, nó đồng thời khóa phương thức get, ngăn không cho Consumer gọi; và ngược lại.

Sử dụng các phương thức notifyAll và wait

❑ Phương thức get :

```
17   public synchronized int get(int who) {  
18      if (!available) {  
19          try { //wait for Producer to put value  
20              wait();  
21          } catch (InterruptedException ie) {  
22              }  
23      }  
24      System.out.println("Customer #" + who + " got: " + contents);  
25      available = false;  
26      //notify Producer that value has been retrieved  
27      notifyAll();  
28      return contents;  
29  }
```

Sử dụng các phương thức notifyAll và wait

❑ Phương thức put:

```
31   public synchronized void put(int who, int value) {  
32      if (available) {  
33          try { //wait for Consumer to get value  
34              wait();  
35          } catch (InterruptedException ie) {  
36              }  
37      }  
38      contents = value;  
39      System.out.println("Producer #" + who + " put: " + contents);  
40      available=true;  
41      //notify Consumer that value has been set  
42      notifyAll();  
43  }  
44 }
```

Các phiên bản khác của `wait`

- ❑ Đây là hai phiên bản của phương thức `wait` chứa trong lớp `Object`:
 - `wait()` : Chờ đợi vô hạn định các thông báo. (phương thức này được dùng trong ví dụ producer-consumer.)
 - `wait(long timeout)` : Chờ thông báo hoặc cho đến khi thời gian *timeout* đã trôi qua. *timeout* được tính theo milliseconds.
 - `wait(long timeout, int nanos)` : Chờ thông báo hoặc cho đến khi thời gian *timeout* cộng *nanos* nanoseconds trôi qua.

Bài tập

BAIO8-BT02: Viết lại **BAIO8-BT01** để 2 đối tượng Producer & Consumer thực hiện đúng yêu cầu.

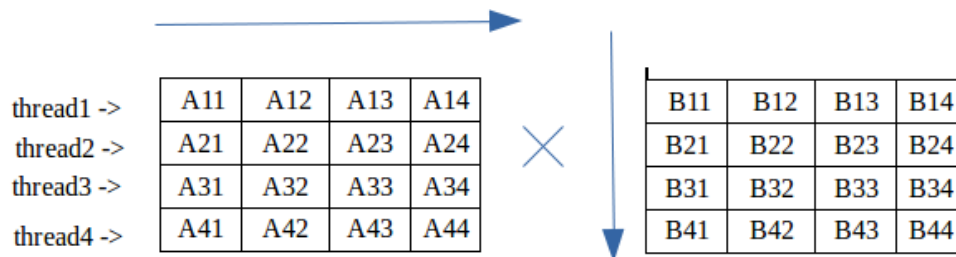
BAIO8-BT03: Viết chương trình tạo 2 tuyến: một tuyến tìm kiếm các số nguyên tố từ 1 đến 1,000,000 và một tuyến tính tổng giá trị của các số nguyên tố tìm được. (Chú ý: có đồng bộ tuyến)

Bài tập

BAI08-BT04: Thực hiện phép nhân 2 ma trận sử dụng đa tuyến.

Mô tả bài toán: phép nhân 2 ma trận là thao tác tốn nhiều thời gian xử lý. Thông thường, độ phức tạp của phép nhân này là $O(n^3)$. Việc áp dụng kỹ thuật đa luồng giúp rút ngắn thời gian xử lý. Thay vì các phép tính được xử lý tuần tự, lúc này ta có nhiều tuyến cùng thực hiện, mỗi tuyến phụ trách một phần của ma trận kết quả.

Ví dụ minh họa:



Ma trận A:

1 0 0
0 1 0
0 0 1

x

Ma Trận B:

2 3 2
4 5 1
7 8 6

=> Output :

Ma trận kết quả:

2 3 2
4 5 1
7 8 6

Yêu cầu: Sinh ngẫu nhiên 2 ma trận A, B có kích thước [1000, 1000], tạo 4 tuyến con để thực hiện phép nhân được mô tả ở trên.

